

8. Object Pool Design Pattern in Java

8.1 Introduction

The **Object Pool Pattern** is a **creational design pattern** that allows reusing objects that are expensive to create. Instead of creating a new object each time, the pool manages a set of pre-instantiated objects.

8.2 Problem it Solves

When object instantiation is expensive in terms of time or resources (e.g., database connections, threads), creating new instances each time can be inefficient.

8.3 What is Object Pool Pattern?

The pattern maintains a pool (set or list) of objects that can be reused instead of creating and destroying them frequently. When a client requests an object, it checks the pool:

- If available, return an idle object
- If not, create a new one (optional, based on pool size)

8.4 Basic Structure

```
package com.mannemlokesesh;
```

```
public interface Poolable {  
    void reset();  
}
```

```
package com.mannemlokesesh;
```

```
import java.util.LinkedList;  
import java.util.Queue;  
import java.util.function.Supplier;  
  
public class ObjectPool<T extends Poolable> {  
    private final Queue<T> availableObjects = new LinkedList<>();  
    private final Supplier<T> creator;  
  
    public ObjectPool(Supplier<T> creator, int initialSize) {  
        this.creator = creator;  
        for (int i = 0; i < initialSize; i++) {  
            availableObjects.offer(creator.get());  
        }  
    }  
  
    public synchronized T borrowObject() {  
        return availableObjects.isEmpty() ?  
            creator.get() : availableObjects.poll();  
    }  
  
    public synchronized void returnObject(T obj) {  
        obj.reset();  
        availableObjects.offer(obj);  
    }  
}
```

8.5 Step-by-Step Examples

Example1: Simple Connection Pool

```
package com.mannemlokesesh;

public class Connection implements Poolable {
    private boolean inUse = false;

    @Override
    public void reset() {
        inUse = false;
    }

    public void connect() {
        inUse = true;
        System.out.println("Connecting to DB... InUse: " + inUse);
    }
}
```

```
package com.mannemlokesesh;

public class Example1ConnectionPoolApp {
    public static void main(String[] args) {
        ObjectPool<Connection> pool = new ObjectPool<>(Connection::new, 2);

        Connection conn1 = pool.borrowObject();
        conn1.connect();

        Connection conn2 = pool.borrowObject();
        conn2.connect();

        pool.returnObject(conn1);
        Connection conn3 = pool.borrowObject();
        conn3.connect();
    }
}
```

Output:

```
Connecting to DB... InUse: true
Connecting to DB... InUse: true
Connecting to DB... InUse: true
```

Example2: Thread Reusable Task Pool

```
package com.mannemlokesesh;

public class Task implements Poolable {
    private String job;

    public void setJob(String job) {
        this.job = job;
    }

    public Runnable getRunnable() {
        String currentJob = this.job; // capture current job safely
        return () -> System.out.println("Processing: " + currentJob);
    }
}
```

```

@Override
public void reset() {
    job = null;
}
}

package com.mannemlokes;

public class Example2TaskPoolApp {
    public static void main(String[] args) {
        ObjectPool<Task> pool = new ObjectPool<>(Task::new, 3);

        Task task1 = pool.borrowObject();
        task1.setJob("Email sending");
        new Thread(task1.getRunnable()).start();

        pool.returnObject(task1);

        Task task2 = pool.borrowObject();
        task2.setJob("Data processing");
        new Thread(task2.getRunnable()).start();
    }
}

```

Output:

```

Processing: Data processing
Processing: Email sending

```

Example3: Custom Reusable Object Manager

```

package com.mannemlokes;

public class ExpensiveResource implements Poolable {
    private int id;
    private static int counter = 1;

    public ExpensiveResource() {
        id = counter++;
        System.out.println("Created ExpensiveResource #" + id);
    }

    @Override
    public void reset() {
        System.out.println("Resetting resource #" + id);
    }

    public void use() {
        System.out.println("Using resource #" + id);
    }
}

```

```

package com.mannemlokes;

public class Example3ResourceApp {
    public static void main(String[] args) {

```

```
ObjectPool<ExpensiveResource> pool =
    new ObjectPool<>(ExpensiveResource::new, 2);

ExpensiveResource res1 = pool.borrowObject();
res1.use();

ExpensiveResource res2 = pool.borrowObject();
res2.use();

pool.returnObject(res1);

ExpensiveResource res3 = pool.borrowObject();
res3.use();
}
```

Output:

```
Created ExpensiveResource #1
Created ExpensiveResource #2
Using resource #1
Using resource #2
Resetting resource #1
Using resource #1
```

8.6 Real-Time Use Cases

- JDBC Connection Pooling
- Thread Pools (ExecutorService)
- Game engine object reuse (bullets, enemies)
- HTTP Client Sessions

8.7 Pros and Cons



Pros

- Reduces resource allocation overhead
- Increases performance
- Reuses existing instances



Cons

- Complexity in pool management
- Not all objects are poolable
- Must manage lifecycle and reset state

8.8 When to Use / Not to Use

Use When

- Object creation is costly
- Short-lived and reusable objects are needed
- Thread-safe resource management is necessary

Avoid When

- Object creation is cheap

- Object state is complex and hard to reset

8.9 Summary

Feature	Description
Reusability	Reuses limited pool of objects
Performance Boost	Avoids expensive instantiation
Pool Management	Requires manual lifecycle handling

The **Object Pool Pattern** is powerful for resource-intensive environments where creating new objects every time isn't feasible. Proper implementation ensures better memory and performance efficiency.